

# LEYAN LI

leyanpower@gmail.com | +86 18805506898 | GitHub: [github.com/leyancode](https://github.com/leyancode) | [leyan-li.com](https://leyan-li.com)

**Work authorisation: eligible for the UK Graduate Route on completion of my MSc — no sponsorship needed to start.**

## EDUCATION

**University of Edinburgh** — MSc High Performance Computing with Data Science | Sep 2025 – Dec 2026

Coursework and dissertation completed Aug 2026 · **available from 1 Sep 2026** · degree conferred Dec 2026

**University of Liverpool** — BSc (Hons) Computer Science, **First-Class Honours** · 2+2 with Xi'an Jiaotong-Liverpool University | Sep 2023 – Jun 2025

**Relevant coursework:** Message-Passing Programming (MPI), Threaded Programming (OpenMP), HPC Architectures, Performance Programming, GPU / Accelerator Programming, Parallel Design Patterns, High-Performance Data Analytics, Machine Learning

## TECHNICAL SKILLS

**Languages:** C / C++, Python, Bash, SQL

**Parallel & performance:** MPI, OpenMP, hybrid MPI+OpenMP, PETSc, Slurm, numactl / hwloc, Linaro MAP; strong / weak scaling, parallel efficiency, saturation points; collective-communication cost (halo exchange, allreduce); roofline and memory-bandwidth analysis; NUMA and thread affinity

**Deep learning & LLM:** PyTorch, NumPy, pandas; LLM function calling and tool-schema design, offline evaluation sets and ablations. GPU and inference serving at coursework / self-study level: CUDA programming model, NCCL, data and tensor parallelism, batching vs. latency trade-offs, KV cache and quantisation

**Engineering:** Linux, Git, pytest, GitHub Actions CI, FastAPI + Pydantic, SQLite / MySQL, working knowledge of Docker

## PROJECTS

**Parallel Layout Selection and Performance Attribution at 4,096 Cores on ARCHER2** | C · PETSc · MPI · OpenMP · Slurm · Linaro MAP · Python / pandas · GitHub Actions [github.com/leyancode/hpc\\_benchmark\\_archer2](https://github.com/leyancode/hpc_benchmark_archer2)  
On ARCHER2, the UK national supercomputer (dual-socket AMD EPYC 7742, 128 cores/node, Slingshot): with total core count fixed, how should work split between MPI ranks and OpenMP threads per rank? Workload was a 2D/3D stencil with CG+GAMG, 5M–165M unknowns, over four full-node layouts (128×1 / 64×2 / 32×4 / 16×8).

- **Layout selection:** In 2D weak scaling, 64×2 beat flat MPI by 13.1% (794.8M equations/s at 165M unknowns on 32 nodes); fixed-20M strong scaling reached 29.83× speedup at 93.2% cumulative efficiency. In 3D the optimum moved to 32×4 (up to +33.2%) and 16×8 (3.81× faster than flat MPI) — the best degree of parallelism shifts with dimension, problem size and per-core work; there is no single “optimal thread count”, and the same diminishing-returns curve governs scaling a fixed batch onto more machines.
- **Root-cause attribution:** Linaro MAP explained the ranking as the crossing point of two curves — as threads per rank rose, MPI share of the solve window fell 46.7% → 6.2% while OpenMP wait time rose 48.3% → 76.5%, trading communication exposure against thread idling. Ruled out threaded BLAS as an explanation (LibSci ≤ 2.1% of solve-window core-time); instrumentation perturbs runtime, so the profile ranked layouts only and was never mixed with uninstrumented timings.
- **Trustworthiness gate:** An audit found 302 of 303 logs whose actual MPI process count disagreed with the filename, so I discarded that entire phase of performance curves; formal conclusions rest on 408 audited runs alone. Made “parse process and thread counts from log content, not filenames” a hard check at the analysis entry point; job scripts are parameterised, pytest pins headline figures, and CI exercises the submission path against a fake Slurm.

**Agent-Based Performance Analysis System over the Benchmark Data** | Python · SQLite · FastAPI · Pydantic · OpenAI SDK (function calling) · pytest [github.com/leyancode/hpc-benchmark-agent](https://github.com/leyancode/hpc-benchmark-agent)

Structured the benchmark data above into SQLite and wrapped each analysis task as a deterministic tool: the LLM only decides which tool to call and with what arguments, while every number comes from deterministic code — turning agent correctness into something measurable.

- **Measurable routing:** Built a rule-based router as a verifiable baseline, output-isomorphic with the LLM function-calling router (intent + arguments), so both score against the same self-built 30-question, five-category evaluation set — 83.3% for rules, 100% for the LLM. An ablation showed the same model scores 63.3% without the system prompt: accuracy is driven mainly by tool documentation and policy prompt, not by the model itself.
- **Defensive parsing and engineering:** Collapsed three failure modes — no tool call, hallucinated tool name, malformed JSON arguments — into an explicit unknown rather than guessing arguments, so the system never returns a silently wrong number. A trace\_id-threaded JSON Lines event stream replays any execution; FastAPI's POST /ask rejects an invalid router as 422 and maps a missing key to 503. 229 automated tests run fully offline.

**Value Factorisation in Multi-Agent Actor-Critic Methods (BSc dissertation)** | Python · PyTorch · PyMARL · MAPPO · VDN · QMIX · CTDE · PPO · PettingZoo / MPE2 [github.com/leyancode/marl-value-factorisation](https://github.com/leyancode/marl-value-factorisation)

- **Counter-intuitive result and root cause:** Attached VDN additive factorisation and QMIX monotonic mixing to a MAPPO critic as a single-variable comparison (variants share runner, buffer, shared-RNN actor and all PPO hyperparameters; only the critic branch differs). The more expressive critic did not win — on sparse-reward Predator-Prey (300k steps) the simplest additive factorisation reached positive return (≈150), the monolithic critic stayed negative, and QMIX's critic loss started at  $5.4 \times 10^4$  and diverged again after 200k steps. The cause: MAPPO-QMIX inherits the value-based TD target  $\max_a Q_i$  (greedy action) while the actor is on-policy, so the resulting td\_error is not a valid advantage but a systematically optimistic quantity. The non-monotonic mixer's persistent gradient explosion is reported as a negative result.

**Publication:** Li, Leyan. Convolutional Neural Networks Based Medical Image Analysis. EMITI 2024.